

Module 0: Notebooks and Values

PUBHLT 0411 · Python for Public Health Data Analysis

Soumik Purkayastha

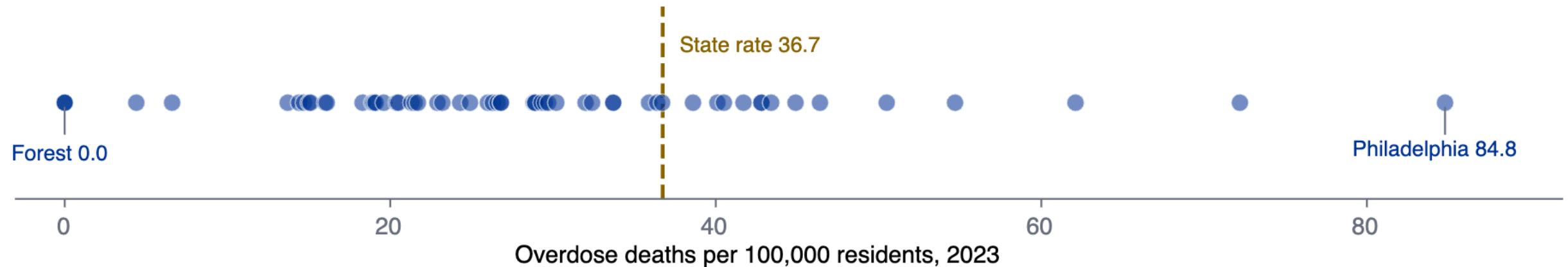
Dept. of Biostatistics and Health Data Science, University of Pittsburgh

October 6, 2026

4,703 people.

The opioid overdose epidemic is the worst public health crisis in Pennsylvania, and the nation, in almost a generation.

Pennsylvania recorded 4,703 overdose deaths in 2023



Source: PA DOH, Estimated Drug Overdose Deaths, 2023

Figure 1. Each dot is one Pennsylvania county's overdose death rate in 2023, plotted on a single axis. The dots spread from Forest County at 0.0 per 100,000 to Philadelphia at 84.8, with most counties clustered between 15 and 40 and the state rate of 36.7 marked by a dashed line.

💡 Today we will discuss....

A state total describes no one. The counties above differ by more than an order of magnitude, and the state figure conceals that variation entirely.

This session establishes what that number means for a single county. The question is answered at the end, from data, using code we will write together.

📖 Prerequisite assumed

Six weeks of R programming experience, but NOTHING else!

The data has a named source and a documented limitation

Every dataset in this course traces to a public agency, and every limitation is stated.

Source	Pennsylvania Department of Health, <i>Estimated Drug Overdose Deaths</i>
Coverage	67 counties, 2019–2024
Definition	Accidental and undetermined drug overdose deaths
Known limitation	PA DOH marks 2024 provisional — death certificates lag three to six months. This session therefore works in 2023 , the most recent complete year of data available
Second limitation	Counts of 1–5 are suppressed , not published. Seven counties are blank in 2023

⚠ Numbers are reductive! In public health we must remember the human beings behind the numbers.

4703 is a quantity admitting arithmetic, and will form the basis of several calculations we do this semester. It is also 4,703 deaths.

The course.

Seven questions to learn about public health in Pennsylvania using Python!

Seven modules: seven questions about Pennsylvania

Each question is answered with code *that we will write*

Module	Area	Skill	Question answered
0	SUBSTANCE USE	Notebook basics and variables	What is Schuylkill County’s overdose death rate?
1	SUBSTANCE USE	Interrogating values, and acting on the answer	Has Centre County met the national overdose target of 13.1 per 100,000?
2	INFECTIOUS DISEASE	Handling many values at once, and code reuse	Which month of the COVID-19 pandemic killed the most Pennsylvanians?
3	ENVIRONMENTAL HEALTH	Whole-dataset arithmetic	How far above the EPA fine-particulate standard is the worst county’s air?
4	NEWBORN HEALTH	Reshaping and combining tables to fit a question	Why do some counties not publish an infant mortality rate?
5	CHRONIC DISEASE	Constructing figures, and describing what they show	Does adult asthma prevalence move with smoking across all 67 counties?
6	INJURY & VIOLENCE	Conducting an analysis end to end, reproducibly	Is the rural firearm death rate really lower than the urban one?

Every dataset originates with the PA Department of Health, the CDC, or the EPA.

 The objective is analytic, not vocational

The purpose of this course is **not** programming proficiency as an end in itself.

The purpose is to pose a question about a population and obtain a defensible answer from data. **Code is instrumental.**

Seven modules: when each one runs

Sessions meet Tuesday and Thursday, 9:30–10:45, in 105 Lawrence Hall.

Module	Area	Lecture	Lab — due 11:59pm	Homework — due 9:00am
0	SUBSTANCE USE	Tue 10/6	Tue 10/6 — <i>not graded</i>	—
1	SUBSTANCE USE	Thu 10/8	Tue 10/13	Thu 10/15
2	INFECTIOUS DISEASE	Thu 10/15	Tue 10/20	Thu 10/22
3	ENVIRONMENTAL HEALTH	Thu 10/22	Tue 10/27	Thu 10/29
4	NEWBORN HEALTH	Thu 10/29	Thu 11/5	Tue 11/10
5	CHRONIC DISEASE	Tue 11/10	Thu 11/12	Tue 11/17
6	INJURY & VIOLENCE	Tue 11/17	Tue 11/17	—

Labs are due the day of the lab session at 11:59pm. Homework is due at 9:00am the morning the next module begins — half an hour before that lecture.

Module 0’s lab is the exception: `module0_lab.ipynb` is a systems check, not an assessment. It is run after class to confirm the environment works, and there is nothing to write and nothing to submit.

 **The Tuesday–Thursday rhythm breaks twice**

Module 4’s lab falls six days after its lecture, spanning Election Day, when the University does not meet.

Module 5 then inverts — lecture Tuesday, lab Thursday. Module 6 is a single combined session.

Assessment weights four components

Component	Weight	Detail
Labs (in class)	40%	Six per module, submitted to Canvas by 11:59pm the day of the lab. No make-ups; lowest per module dropped .
Weekly homework	25%	Five per module, completed on DataCamp , due 9:00am the day the next module starts. Graded on completion. Nothing is dropped .
Final project	30%	In pairs . Teaching session 12/1 or 12/3 (60%), lab packet due exam week (40%).
Participation	5%	Regular attendance and consistent participation.

Office hours: Thursdays, 1–2pm, A740 Crabtree Hall.

 Generative AI is not permitted in this course

The prohibition covers labs, homework, and the final project. It is important to have strong coding skills to understand where AI can produce errors/bugs, verbose code that can be made more efficient, or even hallucinates.

This includes inline code completion. JupyterLab as installed for this course ships no assistant and requires no action. The prohibition applies to any that is added — a Copilot extension, an AI-enabled editor opened alongside, or a browser assistant that completes code in the page.

Substitutes: office hours, lab partners, the class channel, the instructor.

Setup.

Installing Python on your own machine

Six steps, once

The environment is installed locally and takes about fifteen minutes. Steps 1–5 are done once; step 6 confirms the result.

Step	Windows	macOS
1. Open a terminal	PowerShell	Terminal
2. Install uv	one line	one line
3. Unzip the course folder inside PUBHLT0411	identical	identical
4. Point the terminal at the python folder	identical	identical
5. Build the environment	uv sync	uv sync
6. Check that it worked	uv run jupyter lab	uv run jupyter lab

Python is not downloaded separately. Step 5 installs it. Anaconda is not required, and an existing Python or Anaconda installation is left untouched.

 These instructions are on Canvas

SETUP.md carries every step with its failure modes. Nothing here needs to be memorized or copied down.

Step 1 — Open a terminal

A terminal is a window where commands are typed rather than clicked.

Windows	macOS
Start button → type <code>powershell</code> → open Windows PowerShell	<code>Cmd + Space</code> → type <code>terminal</code> → Return
Prompt looks like <code>PS C:\Users\yourname></code>	Prompt looks like <code>yourname@macbook ~ %</code>

Commands are typed at that prompt, exactly as written, each followed by Return.

Step 2 — Install uv

Windows — PowerShell

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

macOS — Terminal

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

About 35 MB, a few seconds, **no administrator rights and no password**.

❗ Then close the terminal and open a new one

The installer changes a setting that only a new window picks up. Confirm with `uv --version`, which should report a version number.

Step 3 — Unzip the course folder inside **PUBHLT0411**

Download **python.zip** from Canvas and **extract it into the **PUBHLT0411** folder already built for the R half**. The result:

```
1 PUBHLT0411/                <- the folder from the R half
2 |— PUBHLT0411.Rproj
3 |— data/                   <- R data
4 |— lecture/ lab/ final-project/
5 |— python/                 <- extracted here
6   |— data/                 <- Python data
7   |— SETUP.md
8   |— module0_lab.ipynb
9   |— pyproject.toml
10  |— uv.lock
```

⚠ Three failures, all common

Opening a zip is not extracting it. Windows displays the contents of a zip without unpacking them, and Python cannot work inside one. Right-click → *Extract All*.

python goes inside **PUBHLT0411, not beside it.** The two halves share one course folder.

Keep **PUBHLT0411 out of OneDrive, Dropbox, and Google Drive.** Syncing software interferes with the files Python creates.

Step 4 — Point the terminal at the `python` folder

Type `cd` and a space, then **drag the `python` folder onto the terminal window** and press Return. Dragging pastes the path correctly, which is faster and more reliable than typing it.

Confirm the location by listing what is there:

Windows	macOS
<code>dir</code>	<code>ls</code>

`pyproject.toml` and a `data` folder should appear in the list. If instead `PUBHLT0411.Rproj` appears, the terminal is one level too high — `cd python` and check again.

Step 5 — Build the environment

```
1 uv sync
```

This installs **Python itself** and every package the course uses. A long list of package names scrolls past; that is expected.

The first run downloads about 200 MB — a minute or two on a good connection, longer on classroom wifi.

 Everyone ends up with identical software

`uv sync` reads a lockfile shipped with the course, so every machine in this room installs the same version of every package. **A result obtained on one laptop is obtainable on any other** — which is what makes reproducibility something the course can assess.

Step 6 — Check that it worked

```
1 uv run jupyter lab
```

JupyterLab opens in the browser. In the file list, open `module0_lab.ipynb` and choose **Run → Run All Cells**.

❗ The notebook prints **SETUP OK**

If it does, the environment is correct and nothing further is required. If it does not, the message states which check failed.

To stop JupyterLab: return to the terminal and press **Ctrl + C** twice.

When something fails

Symptom	Cause and fix
<code>uv</code> is not recognized / command not found	The terminal was not reopened after step 2. Close all terminal windows, open a new one. Restart the computer if it persists.
Windows: <i>running scripts is disabled</i>	The command was shortened. Retype the full line from step 2, including <code>– ExecutionPolicy Bypass</code> .
macOS: <i>operation not permitted</i>	The terminal is denied access to that folder. Move <code>PUBHLT0411</code> to <code>Documents</code> .
<code>uv sync</code> stops partway	The network dropped. Run it again — it resumes rather than restarting.
Failed to spawn: <code>jupyter</code> / program not found	You're not in the <code>python</code> folder. <code>cd</code> into <code>PUBHLT0411/python</code> (check with <code>dir</code> , where you should see <code>pyproject.toml</code>), then <code>uv run jupyter lab</code> again.
No such file or directory: <code>'data/...'</code>	The notebook isn't saved inside the <code>python</code> folder. Save or move it into <code>python/</code> , next to <code>module0_lab.ipynb</code> , and reopen it from JupyterLab.
<code>dir/ls</code> shows <code>PUBHLT0411.Rproj</code>	The terminal is one level too high. <code>cd python</code> .

None of this is assessed. Unresolved problems go to office hours or `soumik@pitt.edu` with a screenshot of the terminal.

A notebook preserves execution state, not screen order

Notebooks contain **code** cells and **text** cells. **Shift+Enter** executes a cell and advances.

The consequential property: **the interpreter retains what was executed, not what is displayed.**

```
1 population = 143786      # executed first
2
3 population = 157795      # executed second
4
5 # The first cell is then edited to 999999 and NOT re-executed.
6 # The value of population remains 157795.
```

Execution order is invocation order

Execution does not proceed top to bottom. The bracketed number beside each cell records the true sequence. This accounts for roughly half of all inexplicable notebook behavior.

Resolution: **Kernel > Restart Kernel and Run All Cells**, which is also the test of whether a notebook is reproducible.

A file is addressed by its path from the `python` folder

The data files ship with the course, inside a `data` folder. Nothing is uploaded.

```
1 PUBHLT0411/
2 └─ python/                               <-- start JupyterLab HERE
3     └─ data/
4         └─ pa_overdose_county_year.csv
5     └─ module0_lab.ipynb
6     └─ module1_lab.ipynb                 <-- each lab saved here as it is posted
7     └─ pyproject.toml
```

A file is then read by its **relative path** — its location starting from the `python` folder:

```
1 pd.read_csv("data/pa_overdose_county_year.csv")
```

⚠ The path is relative to where the notebook is saved

`FileNotFoundError` on a file that plainly exists almost always means the notebook is saved somewhere other than the `python` folder. **Move it into `python/`, reopen it.**

Values.

“The time has come,” the Walrus said, “to talk of many things”.

– Lewis Carroll, Through the Looking-Glass.

Assignment binds a name to a value

The `=` operator does not assert equality. It **binds**: the expression on the right is evaluated, and the name on the left refers to the result.

Python Code ↺ Start Over ▶ Run Code

```
1 county = "Schuylkill"
2 overdose_deaths = 60
3 population = 143786
4
5 print(county)
6 print(overdose_deaths)
```

The statement is read right to left: construct the value, then attach the name.

R analogue

`county <- "Schuylkill"`. The semantics are equivalent. Python uses `=` and provides no `<-`.

Identifier choice is a documentation decision

Python Code

↺ Start Over

▶ Run Code

```
1 # Syntactically valid, semantically opaque.  
2 x = 60  
3 y = 143786  
4  
5 # Syntactically valid, self-documenting.  
6 overdose_deaths = 60  
7 population = 143786  
8  
9 print(overdose_deaths / population * 100000)
```

Constraints: letters, digits, underscores; no spaces; no leading digit. Identifiers are case-sensitive — `Population` and `population` denote different names.

Convention: `lower_case_with_underscores`, not `PopulationCount`.

⚠ Readability is an assessed criterion

Identifier quality falls under presentation, which is a named grading criterion in this course.

Every value carries a type that determines valid operations

Four types account for nearly all data handled in this course.

Python Code ↺ Start Over ▶ Run Code

```
1 county = "Schuylkill"      # str    – text
2 overdose_deaths = 60      # int     – whole number
3 rate_per_100k = 41.7      # float   – decimal number
4 is_rural = True           # bool    – True or False
5
6 print(type(county))
7 print(type(overdose_deaths))
8 print(type(rate_per_100k))
9 print(type(is_rural))
```

`type()` reports the type of any value and is the appropriate diagnostic when an operation behaves unexpectedly.

R analogue

`character`, `integer`, `numeric`, `logical`. Python writes `True` and `False`, not `TRUE` and `FALSE`.

Exercise: bind one county's values

Centre County, 2023: seven deaths, population 157,795.

Exercise

↺ Start Over

💡 Show Hint

▶ Run Code

```
1 county = "Centre"
2 overdose_deaths = _____
3 population = 157795
4
5 print(county, overdose_deaths, population)
```

Python does not coerce between text and numbers

Scenario: a file supplies `"60"` as text. Adding eight should yield `68`.

Python Code

↺ Start Over

▶ Run Code

```
1 overdose_deaths = "60"
2
3 try:
4     print(overdose_deaths + 8)
5 except TypeError as e:
6     print(f"{type(e).__name__}: {e}")
```

⚠ **TypeError** — unsupported operand types

TypeError: can only concatenate str (not "int") to str

The `+` operator is defined for `str + str` (concatenation) and for `int + int` (addition), but not across the two. Rather than infer an intent, the interpreter raises.

Resolution: convert explicitly — `int(overdose_deaths) + 8`.

Conversion between types is explicit

Python Code

↺ Start Over

▶ Run Code

```
1 deaths_as_text = "60"
2
3 print(int(deaths_as_text) + 8)      # 68 – numeric addition
4 print(deaths_as_text + "8")        # 608 – string concatenation
5 print(float("41.7"))               # text to floating point
6 print(str(60) + " deaths")          # numeric to text
7 print(int(41.7))                   # 41 – truncation, not rounding
```

⚠ `int()` truncates toward zero

`int(41.7)` evaluates to `41`. The fractional component is discarded rather than rounded. `round(41.7)` performs rounding and evaluates to `42`.

Seven arithmetic operators

Python Code

↺ Start Over

▶ Run Code

```
1 print(4703 + 67)      # addition
2 print(4703 - 67)      # subtraction
3 print(4703 * 2)       # multiplication
4 print(4703 / 67)      # division      -> float, always
5 print(4703 // 67)     # floor division -> 70, remainder discarded
6 print(4703 % 67)      # modulo        -> 13, the remainder
7 print(2 ** 10)        # exponentiation -> 1024
```

⚠ Three operators warrant particular attention

`/` returns a `float` even when the division is exact. `//` discards the remainder. Exponentiation is `**`; the `^` operator denotes bitwise exclusive-or and **evaluates without error to an incorrect result** when used for powers.

Exercise: compute a rate per 100,000

A rate per 100,000 is the count divided by the population, multiplied by 100,000. Schuylkill County, 2023.

Exercise ↺ Start Over 💡 Show Hint

▶ Run Code

```
1 overdose_deaths = 60
2 population = 143786
3
4 rate_per_100k = overdose_deaths / population * _____
5
6 print(rate_per_100k)
```

i The output carries excessive precision

41.728680121847745 is arithmetically correct and unsuitable for presentation. Formatting is addressed on the following slide.

f-strings interpolate values into text

An `f` prefix on a string literal causes each `{expression}` to be evaluated and substituted.

Python Code ↺ Start Over ▶ Run Code

```
1 county = "Schuylkill"
2 rate_per_100k = 41.72
3
4 print(f"{county} County: {rate_per_100k} per 100,000")
5 print(f"{county} County: {rate_per_100k:.1f} per 100,000")
```

The `:.1f` format specifier fixes the output at one digit after the decimal point; `:.2f` gives two.

 **Format at presentation, retain full precision in computation**

Rounding applied at the point of printing does not affect stored values. Rounding applied during computation propagates through every subsequent step.

Comments document rationale, not syntax

Text following `#` is ignored by the interpreter and addressed to the reader.

Python Code

↺ Start Over

▶ Run Code

```
1 # Redundant: restates the operation.
2 rate_per_100k = 60 / 143786 * 100000 # divide deaths by population
3
4 # Substantive: records a decision about the data.
5 # PA DOH suppresses counts of 1-5. Counties below that threshold are
6 # blank rather than zero; a blank does not indicate the absence of deaths.
7 overdose_deaths = 60
8
9 print(f'{rate_per_100k:.1f}')
```

⚠ Comments are assessed

Interpretation and presentation together account for the majority of the grading criteria. Comments are where analytic reasoning becomes visible to a reader.

One real number.

Answering the question this session opened with

The source file is accessible from this session

`data/pa_overdose_county_year.csv` — 402 observations across 67 counties, 2019–2024.

Python Code

↺ Start Over

▶ Run Code

```
1 ✓ with open("data/pa_overdose_county_year.csv") as handle:
2   for line in handle:
3       print(line.rstrip())
4       break
```

The first line is the header, naming each column.

Python Code

↺ Start Over

▶ Run Code

```
1 ✓ with open("data/pa_overdose_county_year.csv") as handle:
2   lines = handle.readlines()
3
4   print(f"Rows in the file: {len(lines)}")
5   print(lines[1].rstrip())
6   print(lines[2].rstrip())
```

Two counties of comparable population

Schuylkill and Centre, 2023. Both approximately 150,000 residents.

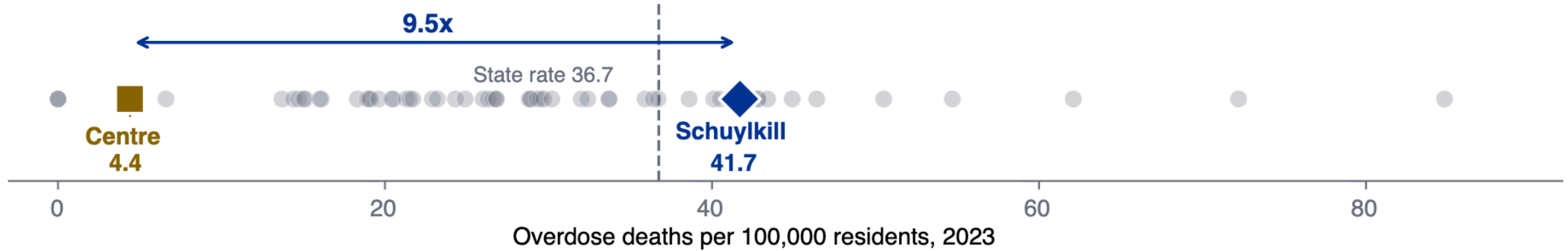
Python Code

↺ Start Over

▶ Run Code

```
1 schuylkill_deaths = 60
2 schuylkill_population = 143786
3
4 centre_deaths = 7
5 centre_population = 157795
6
7 schuylkill_rate = schuylkill_deaths / schuylkill_population * 100000
8 centre_rate = centre_deaths / centre_population * 100000
9
10 print(f"Schuylkill: {schuylkill_rate:.1f} per 100,000")
11 print(f"Centre: {centre_rate:.1f} per 100,000")
12 print(f"Ratio: {schuylkill_rate / centre_rate:.1f}x")
```

The state figure resolves into a ninefold difference



Source: PA DOH, Estimated Drug Overdose Deaths, 2023

Figure 2. The same 59 Pennsylvania counties shown at the start of this session, with two marked. Centre County, a gold square at 4.4 deaths per 100,000, sits near the low end of the distribution; Schuylkill County, a blue diamond at 41.7, sits in the upper third. An arrow spans the two, labelled 9.5x. The state rate of 36.7 is marked by a dashed line. Every other county is greyed.

Comparable populations. A ninefold difference in rate.

! What the state total concealed

4,703 describes Pennsylvania. It resolves into 41.7 in one county and 4.4 in another, and no resident experiences the state average.

Session outcomes

1. Installing the course environment, and verifying it with `module0_lab.ipynb`
2. Executing cells in JupyterLab, and accounting for **execution order as invocation order**
3. Reading a data file by its path relative to the `python` folder
4. Binding values to descriptive identifiers
5. Distinguishing `int`, `float`, `str`, and `bool` via `type()`, and converting between them
6. Applying arithmetic operators, including `//`, `%`, and `**`
7. Formatting output with f-strings and the `:.1f` specifier

Preparation for Thursday

If `module0_lab.ipynb` did not print **SETUP OK** today, resolve it before Thursday — come to my office (A740 Crabtree Hall) and we will troubleshoot it together. Then start JupyterLab in the course folder, open a new notebook, and compute the 2023 overdose rate for one county of interest.

Final exercise: how often?

In 2023, how often did an overdose death occur in each county? Report days and hours.

Exercise [↺ Start Over](#) [💡 Show Hint](#)

[▶ Run Code](#)

```
1 hours = 365 * 24
2
3 # hours between deaths
4 schuylkill_gap = hours // 60
5 s_days = schuylkill_gap // 24
6 s_hours = schuylkill_gap _____ 24
7
8 print(f"Schuylkill: {s_days} d {s_hours} h")
```

💡 Your turn!

Centre recorded 7 deaths. If the true count were 6 or 8, does “ninefold” survive?

💡 Your turn!

If Schuylkill had Centre’s rate, how many deaths would it have had?

That's all folks!

See you on Thursday!